

Formal Translation Rules from WAT to BOOGIE

Mahmoud CHAARI

2025-2026

Contents

1	Formal Translation Rules from WAT to Boogie	2
1.1	Challenges and Notational Conventions	2
1.2	Source Language: WAT	3
1.2.1	Motivation and Characteristics	3
1.2.2	WAT Representation	3
1.2.3	Syntax of the WAT Subset	3
1.2.4	Typing Function	6
1.3	Target Language: Boogie	6
1.3.1	Lexical domains	6
1.3.2	Types	6
1.3.3	Expressions	7
1.3.4	Statements	7
1.3.5	Declarations	7
1.4	Translation Principles	7
1.4.1	Stack Simulation	8
1.4.2	Memory Model	8
1.4.3	Auxiliary Functions and Type Conversions	12
1.4.4	Runtime Initialization	15
1.4.5	Verification Harness Generation	16
1.4.6	Principle of Translation	16
1.5	Formal Translation Rules	17
1.5.1	Constants and Basic Values	17
1.5.2	Numeric & Arithmetic Operations	17
1.5.3	Comparison Operations	20
1.5.4	Bitwise Operations	21
1.5.5	Numeric Conversion Operations	23
1.5.6	Control Flow	24
1.5.7	Memory Operations	26
1.5.8	Table Operations	28
1.5.9	Function Calls and Locals	29
1.5.10	Indirect and Tail Calls	30
1.5.11	Global Variables	32
1.5.12	Imported Functions	32
1.5.13	Global Declarations	32
1.5.14	Exports	33
1.5.15	Verification Harness	33
1.6	Discussion and Scope	34

Chapter 1

Formal Translation Rules from WAT to Boogie

*T*HIS DOCUMENT constitutes the formal core of our translation framework. We define the translation from WebAssembly [1] Text (WAT) into Boogie [2], an intermediate verification language. Unlike previous work such as VeriSol [3], which translates Solidity into Boogie, our case study is more intricate: WAT is a **stack-based language**, while Boogie is **register-based in SSA form**. This semantic mismatch makes the translation significantly more challenging, as implicit data dependencies in the operand stack must be made explicit in Boogie through auxiliary procedures, global variables, helper procedures, and runtime encodings.

The chapter is structured as follows. We begin with the key challenges and notational conventions underlying our formalism. We then formalize the syntax of the WAT subset we consider, followed by the Boogie subset that serves as target language. Next, we present the principles of translation that bridge stack-based semantics and SSA-based representations. Finally, we introduce translation rules for the supported WAT instructions.

1.1 Challenges and Notational Conventions

Two major difficulties arise when defining a translation from WAT to Boogie:

- **Necessity of formal rules.** Translating a programming language into another is not a mere syntactic exercise: It amounts to defining an executable semantics in the target language. To avoid ambiguity, we must give inference-style formal rules that precisely capture how each WAT construct maps to Boogie. These rules ensure the translation is both sound and reproducible.
- **Stack versus register mismatch.** WAT is a *stack-based* language: instructions consume operands from the stack and push results implicitly. Boogie, in contrast, is *register-based and SSA-oriented*, requiring explicit variables, assignments, and control-flow. Bridging this semantic gap requires us to model the stack explicitly in Boogie using global maps and auxiliary procedures.

Notational Conventions

We use the following notations in the inference rules:

- $\vdash$ (turnstile) indicates that the translation is derived under the current typing and environment assumptions.
- $\rightarrow$ (arrow) denotes the translation of a WAT sub-expression or sub-statement into an Boogie code fragment.
- $\hookrightarrow$ (hook arrow) marks the final translation step from a WAT construct to its Boogie encoding.
- The left-hand side of a rule is always a WAT construct, while the right-hand side gives the corresponding Boogie code.

This notation, inspired by VeriSol [3], is here adapted to capture the stack-based execution semantics of WebAssembly.

1.2 Source Language: WAT

1.2.1 Motivation and Characteristics

WebAssembly (WASM) is a portable low-level format, and WAT (WebAssembly Text) is its human-readable form. For our purposes, the key characteristic is that WAT follows a **stack discipline**:

- Each instruction consumes operands from the top of the stack,
- Produces results by pushing them back,
- And relies on structured control constructs with labels.

This stack discipline makes WAT concise, but complicates translation into Boogie, where all data dependencies must be explicit.

1.2.2 WAT Representation

The WebAssembly text format admits two equivalent concrete syntaxes. The first is the canonical *flat* syntax, where instructions are written sequentially according to the stack discipline. The second is the *folded* syntax, which represents nested expressions using S-expressions. Both forms are semantically equivalent and are accepted by standard WebAssembly tools. Throughout this chapter, we assume the canonical flat syntax, as it is the representation generated by `wasm2wat` and the one consumed by our translation framework.

1.2.3 Syntax of the WAT Subset

Lexical domains : We use the following metavariables throughout the grammar and the translation rules. Identifiers are used to name locals, globals, functions, memories, and tables. Labels are used by structured control-flow constructs. Natural numbers appear as indices, offsets, memory sizes, table sizes, and branch depths. String literals are used for import/export annotations.

$x, g, f, m, t \in \text{Identifiers}$	(names of locals, globals, functions, memories, and tables)
$L \in \text{Labels}$	(names of blocks, loops, or branch targets)
$i, j, n \in \mathbb{N}$	(indices, offsets, sizes, and branch depths)
$s \in \text{StringLiterals}$	(strings used for imports and exports)
$e \in \text{InitExpr}$	(constant initialization expression)

Types : WAT distinguishes between integer types (`i32`, `i64`) and floating-point types (`f32`, `f64`). These are the primitive scalar types from which all higher-level values are built.

$$\begin{aligned}
it &\in \text{IntType} ::= \text{i32} \mid \text{i64} \\
ft &\in \text{FloatType} ::= \text{f32} \mid \text{f64} \\
wt &\in \text{WAT-Types} ::= \text{IntType} \cup \text{FloatType}
\end{aligned}$$

Expressions : We separate expressions into producers that push values onto the stack (e.g., constants, arithmetic, memory loads), and consumers/effects that pop values or change state (e.g., assignments, stores, drops). This separation clarifies which instructions contribute to dataflow and which only manipulate control or memory.

Expressions (producers)

$$\begin{aligned}
exp_1 \in \text{WAT-Expressions} ::= & \text{wt.const} \mid \text{local.get } x \mid \text{global.get } x \mid exp_1 \text{ } exp_1 \text{ op} \mid exp_1 \text{ op} \\
& \mid \text{call } f \mid \text{call_indirect } f \mid \text{select} \mid \text{memory.size} \mid \text{memory.grow} \\
& \mid \text{table.get} \mid \text{table.size} \mid \text{table.grow} \\
& \mid \text{it.load} \mid \text{it.load8_s} \mid \text{it.load8_u} \mid \text{it.load16_s} \mid \text{it.load16_u} \\
& \mid \text{i64.load32_s} \mid \text{i64.load32_u} \mid \text{ft.load}
\end{aligned}$$

Expressions (consumers / effects)

$$\begin{aligned}
exp_2 \in \text{WAT-Expressions} ::= & \text{local.set } x \mid \text{local.tee } x \mid \text{global.set } x \mid \text{drop} \mid \text{return} \\
& \mid \text{return_call } f \mid \text{return_call_indirect} \mid \text{br } L \mid \text{br_if } L \\
& \mid \text{br_table} \mid \text{unreachable} \mid \text{memory.copy} \mid \text{memory.fill} \\
& \mid \text{table.set} \\
& \mid \text{it.store} \mid \text{it.store8} \mid \text{it.store16} \mid \text{i64.store32} \mid \text{ft.store}
\end{aligned}$$

Statements : WAT statements combine expressions with control constructs such as `block`, `loop`, and `if`. Unlike typical imperative languages, WAT is structured: all branches and loops are delimited, and control transfer happens via labels instead of unstructured jumps.

$$\begin{aligned}
wst \in \text{WAT-Statements} ::= & exp_1 \mid exp_2 \mid \text{block } L \text{ } wst^* \mid \text{loop } L \text{ } wst^* \\
& \mid \text{if (then } wst^* \text{) (else } wst^* \text{)} \mid \text{if (} wst^* \text{)}
\end{aligned}$$

Operators : Unary and binary operators capture the usual arithmetic, comparison, and logical operations, but are parameterized by type. For example, `i32.add` and `f32.add` both denote addition but apply to different domains.

UnOp ::= `it.eqz` | `i32.wrap_i64` | `it.clz` | `it.ctz` | `it.popcnt`
 | `ft.abs` | `ft.neg` | `ft.sqrt` | `ft.floor` | `ft.ceil` | `ft.trunc` | `ft.nearest`
 | `i64.extend_i32_s` | `i64.extend_i32_u`
 | `it.trunc_ft_s` | `it.trunc_ft_u` | `ft.convert_it_s` | `ft.convert_it_u`
 | `f32.demote_f64` | `f64.promote_f32`

BinOp ::= `wt.add` | `wt.sub` | `wt.mul` | `it.div_s` | `it.div_u` | `ft.div`
 | `it.rem_s` | `it.rem_u`
 | `wt.eq` | `wt.ne` | `it.lt_s` | `it.le_s` | `it.gt_s` | `it.ge_s`
 | `it.lt_u` | `it.le_u` | `it.gt_u` | `it.ge_u` | `ft.lt` | `ft.le` | `ft.gt` | `ft.ge`
 | `it.and` | `it.or` | `it.xor` | `it.shl` | `it.shr_s` | `it.shr_u` | `it.rotl` | `it.rotr`
 | `ft.min` | `ft.max` | `ft.copysign`

$$op \in \text{UnOp} \cup \text{BinOp}$$

Declarations and modules : Functions, locals, and globals are declared at the module level. Imports and exports enable interoperability with the outside world. A WAT module is thus a collection of such declarations, providing the compilation and verification unit for our translation.

FuncDecl ::= `(func f (param x1 wt1 ... xn wtn) (result wtr*) LocalDecl* wt*)`

LocalDecl ::= `(local x wt)`

GlobalDecl ::= `(global x wt e)` | `(global x (mut wt) e)`

MemoryDecl ::= `(memory n)`

TableDecl ::= `(table n funcref)`

ImportDecl ::= `(import s1 s2 (func f (param wt*) (result wt*)))`
 | `(import s1 s2 (memory n))`
 | `(import s1 s2 (table n funcref))`

ExportDecl ::= `(export s (func f))` | `(export s (memory n))` | `(export s (table n))`

Module ::= `(module ImportDecl* GlobalDecl* MemoryDecl* TableDecl* FuncDecl* ExportDecl*)`

Here, *e* denotes a constant initialization expression, typically of the form `(i32.const n)`, `(i64.const n)`, `(f32.const r)`, or `(f64.const r)`. Mutable globals are represented with `(mut wt)`, while immutable globals omit the `mut` keyword.

1.2.4 Typing Function

We use a typing function $\text{type}(e)$ that returns the WebAssembly type of an expression e .

Given a typing environment Γ , the function inspects the syntax of e and determines the type of the value produced on the operand stack. Depending on the expression, the resulting type may be an integer (`i32`, `i64`), a floating-point value (`f32`, `f64`), or a value obtained through comparison, conversion, memory, table, or function-call operations.

Formally,

$$\Gamma \vdash e : wt$$

maps each well-typed WAT expression to the type of the value it leaves on top of the stack.

The typing environment Γ maintains the types of local variables, global variables, function parameters, function results, memories, and tables. It is used whenever the type of an expression cannot be determined syntactically.

The typing function is used throughout the translation rules to select the appropriate Boogie encoding for arithmetic operations, comparisons, conversions, memory accesses, table operations, and function invocations.

This concludes the definition of the WAT subset considered in this work. We have formalized its lexical domains, types, expressions, statements, operators, and module-level constructs. The next section introduces the target language, Boogie, and the subset of Boogie used by SafeWasm for the formal encoding of WebAssembly programs.

1.3 Target Language: Boogie

As recalled in Chapter 2, Boogie is an intermediate verification language used in several verification pipelines. Here we focus on the subset of Boogie that serves as the target of our translation.

1.3.1 Lexical domains

Boogie has a small set of primitive element types used as building blocks for more complex types:

$$bbt \in \text{BoogieElemTypes} ::= \text{int} \mid \text{bool} \mid \text{real}$$

1.3.2 Types

Complex types in Boogie include maps from one domain to another, used to model arrays, memory, or stacks:

$$bt \in \text{BoogieTypes} ::= bbt \mid [bbt]bt$$

Explanation. A type in Boogie is either a primitive element type (`int`, `bool`, `real`) or a mapping type $[bbt]bt$, denoting functions from elements to values. These map types are crucial for modeling memory and stack operations in our translation.

1.3.3 Expressions

Expressions represent values and computations in Boogie:

$$e \in Exprs ::= c \mid x \mid op(e_1, \dots, e_n) \mid uf(e_1, \dots, e_n) \\ \mid x[e] \mid \forall i : bbt :: e$$

Explanation. Expressions include constants c , variables x , applications of built-in operators op , applications of user-defined functions uf , array/mapping lookups $x[e]$, and universally quantified formulas.

1.3.4 Statements

Statements form the executable part of a Boogie program:

$$st \in Stmts ::= \text{skip} \mid \text{havoc} \mid x := e \mid x[e] := e \mid \text{assume } e \mid \text{assert } e \\ \mid \text{if } (e) \{st\} \text{ else } \{st\} \mid st; st \\ \mid \text{while } (e) \text{ do } \{st\} \mid \text{call } \sim x := f(e_1, \dots, e_n)$$

Explanation. Boogie supports imperative-style statements: assignments, assumptions, assertions, sequencing, conditionals, loops, and procedure calls. Verification conditions are ultimately generated from **assert** statements, while **assume** allows the modeler to introduce hypotheses.

1.3.5 Declarations

A Boogie program is a collection of procedure declarations, possibly with specifications:

$$Decl ::= \text{procedure } f(p_1 : bt_1, \dots, p_n : bt_n) \text{ returns } (r_1 : bt_1, \dots, r_k : bt_k) \{st\}$$

Explanation. Each procedure consists of a name, a list of typed input parameters, a list of typed return values, and a body composed of statements. Procedures are the main modular units in Boogie, and serve as the direct translation targets for WAT functions.

This concludes the presentation of the Boogie core language. We have introduced its primitive and composite types, expressions, statements, and procedures, which together form the register-based and SSA-oriented target of our translation. The contrast with WAT is clear: whereas WAT relies on an implicit stack discipline, Boogie requires explicit variables and structured control. The next section develops the principles that connect these two semantic worlds, laying out how stack-based WAT programs can be systematically mapped into verifiable Boogie code.

1.4 Translation Principles

We now explain the general principles that guide the translation from WAT to Boogie. Because WAT is a stack-based language and Boogie is register-based in SSA form, the translation must make all implicit stack operations explicit. This is achieved by modeling the operand

stack, memory, and auxiliary conversions within Boogie. The following subsections describe the simulation model used.

1.4.1 Stack Simulation

The WAT operand stack is represented explicitly in Boogie through a global stack and a stack pointer:

```
$stack : [int] real;    $sp : int;
```

The stack stores all WebAssembly values. For simplicity, every WAT value (`i32`, `i64`, `f32`, and `f64`) is represented as a Boogie value of type `real`.

Three temporary variables are used when extracting values from the stack:

```
$tmp1 : real;    $tmp2 : real;    $tmp3 : real;
```

Stack manipulation is performed through auxiliary procedures:

```
push(val)  ~> $stack[$sp] := val;  $sp := $sp + 1
pop()      ~> $sp := $sp - 1
popToTmp1() ~> $sp := $sp - 1; $tmp1 := $stack[$sp]
popToTmp2() ~> $sp := $sp - 1; $tmp2 := $stack[$sp]
popToTmp3() ~> $sp := $sp - 1; $tmp3 := $stack[$sp]
```

Additional helper procedures `popArgsN` are used when translating function calls. They remove the top N values from the stack and assign them to the corresponding Boogie procedure parameters.

In this way, every WAT instruction that pushes or consumes values on the operand stack can be translated into explicit Boogie statements.

1.4.2 Memory Model

WebAssembly uses a linear byte-addressable memory divided into pages of 64 KiB. SafeWasm represents this memory using the following Boogie variables:

```
$mem : [int]int
```

```
$mem_pages : int
```

The map `$mem` associates each address with the byte stored at that address, while `$mem_pages` stores the current memory size in WebAssembly pages. The total allocated size is therefore

$65536 \times \$mem_pages$.

The intended byte invariant is

$$0 \leq \$\text{mem}[a] < 256.$$

In the current model, this property is enforced locally by the memory helper procedures rather than by a global invariant over the complete memory map.

Byte reads and writes

The procedure `mem_read_u8` reads one byte from address a :

$$result = \$\text{mem}[a].$$

It assumes the following result invariant:

$$0 \leq result < 2^8.$$

The signed procedure `mem_read_s8` interprets the same byte using two's-complement representation:

$$result = \begin{cases} u - 2^8, & \text{if } u \geq 2^7, \\ u, & \text{otherwise,} \end{cases}$$

where u is the value returned by `mem_read_u8`. It therefore produces a value in the signed 8-bit range:

$$-2^7 \leq result < 2^7.$$

The procedure `mem_write_u8` stores a byte at address a :

$$\$ \text{mem}[a] := \text{to_u8}(v).$$

The auxiliary function `to_u8` guarantees only that its result is a valid byte:

$$0 \leq \text{to_u8}(v) < 2^8.$$

Thus, `mem_write_u8` guarantees that the destination cell contains a value between 0 and 255. The exact relationship between v and `to_u8(v)` is left abstract in the current model.

Multi-byte reads

The procedures `mem_read_u16`, `mem_read_u32`, and `mem_read_u64` reconstruct respectively two, four, and eight consecutive bytes using little-endian order.

For an unsigned value of n bytes, the reconstructed result is

$$result = \sum_{i=0}^{n-1} 2^{8i} \times \$\text{mem}[a + i].$$

The unsigned reading procedures guarantee the following ranges:

$$0 \leq \text{mem_read_u16}(a) < 2^{16},$$

$$0 \leq \text{mem_read_u32}(a) < 2^{32},$$

and, provided that all read cells contain valid bytes,

$$0 \leq \text{mem_read_u64}(a) < 2^{64}.$$

The signed procedures `mem_read_s16` and `mem_read_s32` interpret the corresponding unsigned result using two's-complement conversion:

$$\text{signed}_N(u) = \begin{cases} u - 2^N, & \text{if } u \geq 2^{N-1}, \\ u, & \text{otherwise.} \end{cases}$$

They therefore guarantee:

$$-2^{15} \leq \text{mem_read_s16}(a) < 2^{15},$$

$$-2^{31} \leq \text{mem_read_s32}(a) < 2^{31}.$$

The current implementation of `mem_read_s64` directly returns the result of `mem_read_u64`. Therefore, signed 64-bit interpretation is not yet modeled precisely.

Multi-byte writes

The procedures `mem_write_u16`, `mem_write_u32`, and `mem_write_u64` write respectively two, four, and eight bytes in little-endian order.

For an n -byte write, the intended effect is

$$\text{\$mem}[a + i] := \text{byte}_i(v), \quad 0 \leq i < n.$$

Each helper function `byte_i` is constrained by the invariant

$$0 \leq \text{byte}_i(v) < 2^8.$$

Therefore, the procedures guarantee that each modified memory cell contains a valid byte. However, the current model does not yet formally specify that each `byte_i(v)` is the exact i -th byte of v .

Memory size and growth

The procedure `memory_size` returns the current number of allocated pages:

$$\text{result} = \text{\$mem_pages}.$$

It does not modify either the memory contents or the page count.

The procedure `memory_grow` returns the previous number of pages and increases the page count by δ :

$$oldSize = old(\$mem_pages),$$

$$\$mem_pages = old(\$mem_pages) + \delta.$$

The memory contents remain unchanged. The current model assumes that growth always succeeds and does not model the WebAssembly failure result -1 , maximum memory limits, or zero-initialization of newly allocated pages.

Bulk-memory operations

The procedure `memory_fill` requires a non-negative length:

$$len \geq 0.$$

It guarantees that every byte in the destination interval $[dst, dst+len)$ is set to `to_u8(value)`:

$$\forall i : int . 0 \leq i < len \implies \$mem[dst + i] = to_u8(value).$$

It also guarantees that all memory cells outside the destination interval remain unchanged:

$$\forall a : int . a < dst \vee a \geq dst + len \implies \$mem[a] = old(\$mem[a]).$$

The procedure `memory_copy` also requires

$$len \geq 0.$$

It guarantees that every destination byte receives the corresponding byte from the old memory state:

$$\forall i : int . 0 \leq i < len \implies \$mem[dst + i] = old(\$mem[src + i]).$$

The use of `old` ensures that overlapping source and destination intervals are modeled correctly. As for `memory_fill`, memory cells outside the destination interval remain unchanged:

$$\forall a : int . a < dst \vee a \geq dst + len \implies \$mem[a] = old(\$mem[a]).$$

Current limitations

The current memory model does not explicitly check that load, store, fill, or copy operations remain within the allocated memory. Therefore, out-of-bounds accesses and the corresponding WebAssembly traps are not yet modeled.

In addition, the helper functions `to_u8` and `byte_i` currently guarantee only that their results belong to the byte range. Their exact arithmetic relationship with the original value remains abstract.

1.4.3 Auxiliary Functions and Type Conversions

SafeWasm currently represents all WebAssembly stack values using the Boogie type `real`. Auxiliary functions are therefore introduced to model Boolean values, numeric conversions, mathematical operations, and integer-specific instructions.

These helpers have different levels of precision. Some are defined directly by a Boogie expression, some are constrained by axioms, while others remain uninterpreted in the current model.

Boolean conversions

Boolean values are encoded on the operand stack using the following functions:

$$\text{bool_to_real}(b) = \begin{cases} 1.0, & \text{if } b = \text{true}, \\ 0.0, & \text{otherwise,} \end{cases}$$

and

$$\text{real_to_bool}(r) = \begin{cases} \text{false}, & \text{if } r = 0.0, \\ \text{true}, & \text{otherwise.} \end{cases}$$

The function `bool_to_real` guarantees that its result is either 0.0 or 1.0:

$$\forall b : \text{bool} . \text{bool_to_real}(b) = 0.0 \vee \text{bool_to_real}(b) = 1.0.$$

The two conversion functions are related by the following invariant:

$$\forall b : \text{bool} . \text{real_to_bool}(\text{bool_to_real}(b)) = b.$$

Moreover,

$$\forall r : \text{real} . \text{real_to_bool}(r) = \text{false} \iff r = 0.0.$$

These properties allow comparison results to be stored on the real-valued operand stack and later reused as branch conditions.

Numeric conversions

SafeWasm declares the following conversion functions:

$$\begin{aligned}
\text{real_to_int} & : \text{real} \rightarrow \text{int}, \\
\text{int_to_real} & : \text{int} \rightarrow \text{real}, \\
\text{bits32_to_real} & : \text{int} \rightarrow \text{real}, \\
\text{bits64_to_real} & : \text{int} \rightarrow \text{real}.
\end{aligned}$$

These functions are used when translating memory operations, integer instructions, and reinterpretation instructions.

In the current prelude, they are uninterpreted functions. Consequently, Boogie knows their input and output types and assumes that they are deterministic, but no arithmetic relationship between their inputs and outputs is currently guaranteed.

For example, the current model does not yet establish

$$\text{real_to_int}(\text{int_to_real}(i)) = i.$$

Such properties would require additional definitions or axioms.

Basic real-valued operations

The functions `min_real`, `max_real`, and `abs_real` are defined directly in Boogie.

The minimum and maximum functions satisfy

$$\text{min_real}(x, y) = \begin{cases} x, & \text{if } x \leq y, \\ y, & \text{otherwise,} \end{cases}$$

and

$$\text{max_real}(x, y) = \begin{cases} x, & \text{if } x \geq y, \\ y, & \text{otherwise.} \end{cases}$$

The absolute-value function is defined as

$$\text{abs_real}(x) = \begin{cases} x, & \text{if } x \geq 0.0, \\ -x, & \text{otherwise.} \end{cases}$$

These functions therefore have an explicit and precise mathematical semantics in the current model.

Axiomatized mathematical functions

The function `sqrt_real` is left uninterpreted but constrained by the following axioms for non-negative inputs:

$$r \geq 0.0 \implies \text{sqrt_real}(r) \geq 0.0,$$

and

$$r \geq 0.0 \implies \text{sqrt_real}(r)^2 = r.$$

No behavior is specified for negative inputs.

The function `nearest_real` is constrained so that its result lies within a distance of 0.5 from the input:

$$\text{nearest_real}(r) - 0.5 \leq r \leq \text{nearest_real}(r) + 0.5.$$

It is also idempotent:

$$\text{nearest_real}(\text{nearest_real}(r)) = \text{nearest_real}(r).$$

These axioms provide only a partial abstraction of WebAssembly's nearest-rounding semantics. In particular, they do not specify the tie-breaking rule.

The function `floor_real` is constrained by

$$\text{floor_real}(r) \leq r$$

and

$$r < \text{floor_real}(r) + 1.0.$$

These inequalities place the result in the interval

$$r - 1.0 < \text{floor_real}(r) \leq r.$$

However, the current model does not explicitly require `floor_real(r)` to represent an integer-valued real.

Uninterpreted mathematical functions

The following functions are currently declared without definitions or axioms:

$$\text{ceil_real}, \quad \text{trunc_real}, \quad \text{copysign_real}.$$

Therefore, their exact mathematical semantics is not yet modeled. Boogie treats them as deterministic but otherwise arbitrary functions.

Bitwise and integer operations

SafeWasm declares the following functions for bitwise operations:

$$\begin{aligned} &\text{bv_and}, \quad \text{bv_or}, \quad \text{bv_xor}, \\ &\text{bv_shl}, \quad \text{bv_shr_s}, \quad \text{bv_shr_u}, \\ &\text{bv_rotr}, \quad \text{bv_rotr}. \end{aligned}$$

The following integer-specific operations are also declared:

`int_rem`, `int_clz`, `int_ctz`, `int_popcnt`.

All these functions currently have type

$$real \times real \rightarrow real$$

or

$$real \rightarrow real,$$

depending on their arity.

They are currently uninterpreted. Consequently, the model does not yet guarantee standard bit-vector properties such as

$$bv_and(x, 0) = 0,$$

$$bv_xor(x, x) = 0,$$

or the expected range of `int_clz`, `int_ctz`, and `int_popcnt`.

At the current stage, these declarations allow the corresponding WebAssembly instructions to be translated into Boogie, but they do not yet provide a complete semantic model of these operations.

Current abstraction level

The current auxiliary-function model can therefore be summarized as follows:

- Boolean conversions and the functions `min_real`, `max_real`, and `abs_real` have explicit definitions.
- The functions `sqrt_real`, `nearest_real`, and `floor_real` have partial semantics expressed through axioms.
- Numeric conversions, remaining rounding operations, bitwise operations, and integer-specific operations are currently uninterpreted.

Further refinements may introduce additional axioms, explicit function definitions, or native Boogie bit-vector encodings when stronger verification guarantees are required.

1.4.4 Runtime Initialization

Before executing the translated body of a WebAssembly function, SafeWasm initializes the Boogie runtime state. This initialization is performed through two auxiliary procedures: `InitRuntime` and `initGlobals`.

Explanation. The procedure `InitRuntime` initializes the execution stack and the runtime variables used by the translation, such as the stack pointer `$sp`. The procedure `initGlobals`

initializes mutable global variables and memory-related runtime state according to the declarations of the original WebAssembly module. Immutable globals are instead translated as Boogie constants, constrained by axioms when their initialization value is known statically.

This initialization step ensures that the generated Boogie program starts from a runtime state consistent with the source WebAssembly module before executing the translated function body.

1.4.5 Verification Harness Generation

In addition to translating WebAssembly functions into Boogie procedures, SafeWasm generates verification harnesses that define executable entry points for the verification backends. These harnesses are not translations of WebAssembly instructions. Instead, they are generated around the translated module in order to drive the verification process.

The main Boogie harness initializes the runtime state and the module state by calling `initGlobals` and `InitRuntime`. It then executes an unbounded loop in which one function of the translated WebAssembly module is selected non-deterministically. The selected function is called with non-deterministically generated arguments, which are pushed onto the simulated operand stack before the call.

For integer parameters, the harness constrains the generated value to be integer-like in the current real-valued encoding. If the called function returns values, these values are removed from the stack after the call, since the harness is interested in checking the preservation of the global verification state rather than using a particular return value.

This harness therefore models arbitrary sequences of public function calls over the translated module. It is mainly used to verify that the generated Boogie model preserves the specified global invariants across all possible function transitions.

1.4.6 Principle of Translation

The translation follows these general principles:

- **Expressions.** Every WAT expression e is translated into a Boogie sequence that leaves its result on the stack via `push`.

$$e \rightarrow \chi \quad \text{with } \chi \text{ ending in } \text{push}(\text{val})$$

- **Statements.** WAT statements are translated into structured Boogie statements (`if`, `while`, `goto`), with labels used to model WAT's control-flow blocks.

$$wst \rightarrow S \quad \text{where } S \text{ is a Boogie statement/block}$$

- **Function Calls.** A WAT call pushes arguments on the stack, and the callee uses a helper procedure `popArgsN` at the start of its body to bind them to parameters.

With these principles in place, the translation rules of the next section can be read systematically: stack effects in WAT become explicit stack manipulations in Boogie, and structured control is preserved through Boogie's imperative constructs.

1.5 Formal Translation Rules

We now present the core translation rules from WAT into Boogie. For readability, we include here only representative rules from each category (constants, arithmetic, control flow, memory, and functions). The complete set of rules is deferred to Appendix. Each rule is written in inference style: the premise above the line indicates the translation of sub-expressions, and the conclusion gives the corresponding Boogie code.

1.5.1 Constants and Basic Values

$$\frac{type(n) = it}{\vdash \text{it.const } n \hookrightarrow \text{push(int_to_real}(n))} (\text{CONST-INT})$$

$$\frac{type(n) = ft}{\vdash \text{ft.const } n \hookrightarrow \text{push}(n)} (\text{CONST-FLOAT})$$

Explanation. Integer constants are converted into `real` values via `int_to_real`, while floating-point constants are directly pushed onto the Boogie stack.

1.5.2 Numeric & Arithmetic Operations

$$\frac{e_1 \in exp_1 \quad e_2 \in exp_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_1; \chi_2;} (\text{ADD})$$

`\vdash e_1 e_2 wt.add \hookrightarrow` `call popToTmp1();`
`call popToTmp2();`
`call push($tmp2 + $tmp1);`

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_2; \chi_1;} (\text{SUB})$$

`\vdash e_1 e_2 wt.sub \hookrightarrow` `call popToTmp1();`
`call popToTmp2();`
`call push($tmp2 - $tmp1);`

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_2; \chi_1;} (\text{MUL})$$

`\vdash e_1 e_2 wt.mul \hookrightarrow` `call popToTmp1();`
`call popToTmp2();`
`call push($tmp2 * $tmp1);`

$$\frac{op \in \{\text{div_s}, \text{div_u}\} \quad e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\text{DIV_S, DIV_U}}$$

$\chi_2;$

$\chi_1;$

$\vdash e_1 \ e_2 \text{ it.op} \hookrightarrow \text{call popToTmp1}();$

$\text{call popToTmp2}();$

$\text{call push}(\$tmp2 / \$tmp1);$

$$\frac{e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\text{(DIV)}}$$

$\chi_2;$

$\chi_1;$

$\vdash e_1 \ e_2 \text{ ft.div} \hookrightarrow \text{call popToTmp1}();$

$\text{call popToTmp2}();$

$\text{call push}(\$tmp2 / \$tmp1);$

$$\frac{op \in \{\text{rem_s}, \text{rem_u}\} \quad e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\text{(REM_S, REM_U)}}$$

$\chi_2;$

$\chi_1;$

$\vdash e_1 \ e_2 \text{ it.op} \hookrightarrow \text{call popToTmp1}();$

$\text{call popToTmp2}();$

$\text{call push}(\text{int_rem}(\$tmp2, \$tmp1));$

$$\frac{e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\text{(MIN)}}$$

$\chi_2;$

$\chi_1;$

$\vdash e_1 \ e_2 \text{ ft.min} \hookrightarrow \text{call popToTmp1}();$

$\text{call popToTmp2}();$

$\text{call push}(\text{min_real}(\$tmp2, \$tmp1));$

$$\frac{e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\text{(MAX)}}$$

$\chi_2;$

$\chi_1;$

$\vdash e_1 \ e_2 \text{ ft.max} \hookrightarrow \text{call popToTmp1}();$

$\text{call popToTmp2}();$

$\text{call push}(\text{max_real}(\$tmp2, \$tmp1));$

$$\begin{array}{c}
\frac{e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_2; \quad \chi_1;} \text{(COPYSIGN)} \\
\vdash e_1 \ e_2 \text{ ft.copysign} \hookrightarrow \text{call popToTmp1()}; \\
\quad \text{call popToTmp2()}; \\
\quad \text{call push(copysign_real(\$tmp2, \$tmp1))}; \\
\\
\frac{e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\chi;} \text{(ABS)} \\
\vdash \text{ft.abs } e \hookrightarrow \text{call popToTmp1()}; \\
\quad \text{call push(abs_real(\$tmp1))}; \\
\\
\frac{e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\chi;} \text{(NEG)} \\
\vdash \text{ft.neg } e \hookrightarrow \text{call popToTmp1()}; \\
\quad \text{call push(-\$tmp1)}; \\
\\
\frac{e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\chi;} \text{(SQRT)} \\
\vdash \text{ft.sqrt } e \hookrightarrow \text{call popToTmp1()}; \\
\quad \text{call push(sqrt_real(\$tmp1))}; \\
\\
\frac{e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\chi;} \text{(FLOOR)} \\
\vdash \text{ft.floor } e \hookrightarrow \text{call popToTmp1()}; \\
\quad \text{call push(floor_real(\$tmp1))}; \\
\\
\frac{e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\chi;} \text{(CEIL)} \\
\vdash \text{ft.ceil } e \hookrightarrow \text{call popToTmp1()}; \\
\quad \text{call push(ceil_real(\$tmp1))}; \\
\\
\frac{e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\chi;} \text{(TRUNC)} \\
\vdash \text{ft.trunc } e \hookrightarrow \text{call popToTmp1()}; \\
\quad \text{call push(trunc_real(\$tmp1))}; \\
\\
\frac{e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\chi;} \text{(NEAREST)} \\
\vdash \text{ft.nearest } e \hookrightarrow \text{call popToTmp1()}; \\
\quad \text{call push(nearest_real(\$tmp1))};
\end{array}$$

$$\frac{e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\chi;} \text{---(CLZ)}$$

$\vdash e \text{ it.clz} \hookrightarrow \text{call popToTmp1()};$
 $\text{call push(int_clz(\$tmp1));}$

$$\frac{e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\chi;} \text{---(CTZ)}$$

$\vdash e \text{ it.ctz} \hookrightarrow \text{call popToTmp1()};$
 $\text{call push(int_ctz(\$tmp1));}$

$$\frac{e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\chi;} \text{---(CLZ)}$$

$\vdash e \text{ it.popcnt} \hookrightarrow \text{call popToTmp1()};$
 $\text{call push(int_popcnt(\$tmp1));}$

1.5.3 Comparison Operations

$$\frac{e \in \text{exp_1} \quad \vdash e \rightarrow \chi}{\chi;} \text{---(EQZ)}$$

$\vdash \text{it.eqz } e \hookrightarrow \text{call popToTmp1()};$
 $\text{call push(bool_to_real(\$tmp1 == 0.0));}$

$$\frac{e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_2;} \text{---(EQ)}$$

$\chi_1;$

$\vdash e_1 \ e_2 \text{ wt.eq} \hookrightarrow \text{call popToTmp1()};$
 $\text{call popToTmp2()};$
 $\text{call push(bool_to_real(\$tmp2 == \$tmp1));}$

$$\frac{e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_2;} \text{---(NE)}$$

$\chi_1;$

$\vdash e_1 \ e_2 \text{ wt.ne} \hookrightarrow \text{call popToTmp1()};$
 $\text{call popToTmp2()};$
 $\text{call push(bool_to_real(\$tmp2 != \$tmp1));}$

$$\begin{array}{c}
\frac{op \in \{\text{it.lt_s}, \text{it.lt_u}, \text{ft.lt}\} \quad e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\text{LT}} \\
\chi_2; \\
\chi_1; \\
\vdash e_1 \ e_2 \ op \hookrightarrow \text{call popToTmp1()}; \\
\text{call popToTmp2()}; \\
\text{call push(bool_to_real(\$tmp2 < \$tmp1))}; \\
\frac{op \in \{\text{it.le_s}, \text{it.le_u}, \text{ft.le}\} \quad e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\text{LE}} \\
\chi_2; \\
\chi_1; \\
\vdash e_1 \ e_2 \ op \hookrightarrow \text{call popToTmp1()}; \\
\text{call popToTmp2()}; \\
\text{call push(bool_to_real(\$tmp2 <= \$tmp1))}; \\
\frac{op \in \{\text{it.gt_s}, \text{it.gt_u}, \text{ft.gt}\} \quad e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\text{GT}} \\
\chi_2; \\
\chi_1; \\
\vdash e_1 \ e_2 \ op \hookrightarrow \text{call popToTmp1()}; \\
\text{call popToTmp2()}; \\
\text{call push(bool_to_real(\$tmp2 > \$tmp1))}; \\
\frac{op \in \{\text{it.ge_s}, \text{it.ge_u}, \text{ft.ge}\} \quad e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\text{GE}} \\
\chi_2; \\
\chi_1; \\
\vdash e_1 \ e_2 \ op \hookrightarrow \text{call popToTmp1()}; \\
\text{call popToTmp2()}; \\
\text{call push(bool_to_real(\$tmp2 >= \$tmp1))};
\end{array}$$

1.5.4 Bitwise Operations

$$\begin{array}{c}
\frac{e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\text{AND}} \\
\chi_2; \\
\chi_1; \\
\vdash e_1 \ e_2 \ \text{it.and} \hookrightarrow \text{call popToTmp1()}; \\
\text{call popToTmp2()}; \\
\text{call push(bv_and(\$tmp2, \$tmp1))};
\end{array}$$

$$\begin{array}{c}
\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_2; \quad \chi_1;} \text{(OR)} \\
\vdash e_1 \ e_2 \ it.or \hookrightarrow \text{call popToTmp1()}; \\
\text{call popToTmp2()}; \\
\text{call push(bv_or(\$tmp2, \$tmp1))}; \\
\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_2; \quad \chi_1;} \text{(XOR)} \\
\vdash e_1 \ e_2 \ it.xor \hookrightarrow \text{call popToTmp1()}; \\
\text{call popToTmp2()}; \\
\text{call push(bv_xor(\$tmp2, \$tmp1))}; \\
\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_2; \quad \chi_1;} \text{(SHL)} \\
\vdash e_1 \ e_2 \ it.shl \hookrightarrow \text{call popToTmp1()}; \\
\text{call popToTmp2()}; \\
\text{call push(bv_shl(\$tmp2, \$tmp1))}; \\
\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_2; \quad \chi_1;} \text{(SHR_S)} \\
\vdash e_1 \ e_2 \ it.shr_s \hookrightarrow \text{call popToTmp1()}; \\
\text{call popToTmp2()}; \\
\text{call push(bv_shr_s(\$tmp2, \$tmp1))}; \\
\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_2; \quad \chi_1;} \text{(SHR_U)} \\
\vdash e_1 \ e_2 \ it.shr_u \hookrightarrow \text{call popToTmp1()}; \\
\text{call popToTmp2()}; \\
\text{call push(bv_shr_u(\$tmp2, \$tmp1))};
\end{array}$$

$$\begin{array}{c}
\frac{e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_2; \quad \chi_1;} \text{---(ROTL)} \\
\vdash e_1 \ e_2 \ \text{it.rotl} \hookrightarrow \text{call popToTmp1()}; \\
\text{call popToTmp2()}; \\
\text{call push(bv_rotl(\$tmp2, \$tmp1))}; \\
\frac{e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\chi_2; \quad \chi_1;} \text{---(ROTR)} \\
\vdash e_1 \ e_2 \ \text{it.rotr} \hookrightarrow \text{call popToTmp1()}; \\
\text{call popToTmp2()}; \\
\text{call push(bv_rotr(\$tmp2, \$tmp1))};
\end{array}$$

Explanation. Bitwise operations consume two integer operands from the WAT operand stack. SafeWasm evaluates the two operands, pops them into `$tmp1` and `$tmp2`, and delegates the corresponding bit-level semantics to dedicated Boogie functions such as `bv_and`, `bv_or`, `bv_shl`, and `bv_shr_u`. The result is then pushed back onto the stack as a `real` value.

1.5.5 Numeric Conversion Operations

WebAssembly provides several instructions to convert values between integer and floating-point types, or between values of different sizes. Since the Boogie operand stack stores only values of type `real`, these conversions are implemented through auxiliary conversion functions defined in the runtime prelude.

$$\begin{array}{c}
\frac{op \in \{\text{i64.extend_i32_s}, \text{i64.extend_i32_u}\} \quad e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\vdash e \ op \hookrightarrow \chi; \quad // \text{ No conversion required}} \text{---(EXTEND)} \\
\frac{e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\vdash e \ \text{i32.wrap_i64} \hookrightarrow \chi; \quad // \text{ No conversion required}} \text{---(WRAP)} \\
\frac{op \in \{\text{f64.promote_f32}, \text{f32.demote_f64}\} \quad e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\vdash e \ op \hookrightarrow \chi; \quad // \text{ No conversion required}} \text{---(FLOAT-CONVERSION)}
\end{array}$$

$$\frac{op \in \{\text{it.reinterpret_ft}, \text{ft.reinterpret_it}\} \quad e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\vdash e \rightarrow \chi} \text{(REINTERPRET)}$$

$\vdash e \rightarrow \chi$;
// No conversion required

$$\frac{op \in \{\text{it.trunc_ft_s}, \text{it.trunc_ft_u}\} \quad e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\vdash e \rightarrow \chi} \text{(TRUNC)}$$

$$\frac{op \in \{\text{ft.convert_it_s}, \text{ft.convert_it_u}\} \quad e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\vdash e \rightarrow \chi} \text{(CONVERT)}$$

1.5.6 Control Flow

$$\frac{\frac{wst \rightarrow S}{\text{call popToTmp1();}} \text{(IF)} \quad \vdash \text{if (then } wst) \hookrightarrow \begin{array}{l} \text{if (real_to_bool(\$tmp1)) \{ } \\ S \\ \} \end{array}}{\vdash \text{if (then } wst_1) \text{ (else } wst_2) \hookrightarrow \begin{array}{l} S_1 \\ \} \text{ else \{ } \\ S_2 \\ \} \end{array}} \text{(IF ELSE)}$$

$$\frac{\vdash wst_1 \rightarrow S_1 \quad \vdash wst_2 \rightarrow S_2}{\vdash \text{if (result } wt) \text{ (then } wst_1) \text{ (else } wst_2) \hookrightarrow \begin{array}{l} \text{call popToTmp1();} \\ \text{if (real_to_bool(\$tmp1)) \{ } \\ S_1; \\ \text{call popToTmp2();} \\ \$tmp3 := \$tmp2; } \\ S_2; \\ \text{call popToTmp2();} \\ \$tmp3 := \$tmp2; \\ \} \\ \text{call push(\$tmp3);} \end{array}} \text{(IF-ELSE-RESULT)}$$

Explanation. The condition of a WebAssembly `if` is evaluated before the `if` instruction itself. Therefore, when translating the `if/else` construct, SafeWasm assumes that the condition is already available on top of the operand stack and retrieves it using `popToTmp1()`.

$$\begin{array}{c}
\frac{}{\vdash \text{br } L \hookrightarrow \text{goto } L;} \text{(BR)} \\
\\
\frac{}{\vdash \text{br_if } L \hookrightarrow \begin{array}{l} \text{call popToTmp1();} \\ \text{if (real_to_bool(\$tmp1)) \{ } \\ \quad \text{goto } L; \\ \quad \text{\}} \end{array}} \text{(BR_IF)} \\
\\
\frac{\vdash wst \rightarrow S}{\vdash \text{block } L \text{ } wst \text{ end } \hookrightarrow \begin{array}{l} S \\ \text{label } L_{\text{end}}: \end{array}} \text{(BLOCK)} \\
\\
\frac{\vdash wst \rightarrow S}{\vdash \text{block } L \text{ (result } wt) \text{ } wst \text{ end } \hookrightarrow \begin{array}{l} S; \\ \text{label } L_{\text{end}}: \end{array}} \text{(BLOCK-RESULT)}
\end{array}$$

Explanation. In WebAssembly, a `block` defines a structured control-flow region whose branch target is its end rather than its beginning. Consequently, SafeWasm translates each block by generating a label corresponding to the block exit. Instructions inside the block are translated normally, and any `br` instruction targeting the block is translated into a `goto` directed to this end label. If the block has a result type, the corresponding value remains on the operand stack when execution reaches the end label, thereby preserving the semantics of WebAssembly block expressions.

$$\begin{array}{c}
\frac{wst \rightarrow S}{\text{label } L:} \text{(LOOP)} \\
\\
\vdash \text{loop } L \text{ } wst \hookrightarrow \begin{array}{l} S \\ \text{goto } L; \end{array}
\end{array}$$

$$\begin{array}{c}
\frac{e_1, e_2, e_3 \in exp_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2 \quad \vdash e_3 \rightarrow \chi_3}{\text{(SELECT)}} \\
\begin{array}{l}
\chi_1; \\
\chi_2; \\
\chi_3; \\
\text{call popToTmp1}(); \\
\text{call popToTmp2}(); \\
\vdash e_1 \ e_2 \ e_3 \ \text{select} \hookrightarrow \text{call popToTmp3}(); \\
\quad \text{if (real_to_bool(\$tmp1)) \{ } \\
\quad \quad \text{call push(\$tmp3);} \\
\quad \text{\} else \{ } \\
\quad \quad \text{call push(\$tmp2);} \\
\quad \text{\} } \\
\end{array} \\
\frac{}{\vdash \text{end} \hookrightarrow \epsilon} \quad \text{(END)} \\
\frac{}{\vdash \text{nop} \hookrightarrow \epsilon} \quad \text{(NOP)}
\end{array}$$

Explanation. The symbol ϵ denotes the empty sequence of Boogie commands.

$$\frac{}{\vdash \text{drop} \hookrightarrow \text{pop}();} \text{(DROP)}$$

$$\frac{}{\vdash \text{unreachable} \hookrightarrow \text{assert false};} \text{(UNREACHABLE)}$$

Explanation. Structured control flow in WAT is mapped to explicit Boogie constructs: loops use labels and gotos, while conditionals rely on popping the top of the stack and evaluating it as a Boolean.

1.5.7 Memory Operations

$$\frac{load_i \in BoogieLocalVars}{\vdash \text{memory.size} \hookrightarrow \begin{array}{l} \text{call load_i := memory_size}; \\ \text{call push(int_to_real(load_i));} \end{array}} \text{(MEMORY.SIZE)}$$

$$\frac{\text{load}_i \in \text{BoogieLocalVars} \quad e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\vdash e \text{ memory.grow} \hookrightarrow \begin{array}{l} \chi; \\ \text{call popToTmp1}(); \\ \text{call load_i} := \text{memory_grow}(\text{real_to_int}(\$tmp1)); \\ \text{call push}(\text{int_to_real}(\text{load_i})); \end{array}} \text{(MEMORY.GROW)}$$

$$\frac{\text{idx}, \text{load}_i \in \text{BoogieLocalVars} \quad \text{load} \in \{\text{it.load}, \text{ft.load}\} \quad e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\vdash e \text{ load} \hookrightarrow \begin{array}{l} \chi; \\ \text{call popToTmp1}(); \\ \text{idx} := \text{real_to_int}(\$tmp1); \\ \text{call load_i} := \text{mem_read}(\text{idx}); \\ \text{call push}(\text{int_to_real}(\text{load_i})); \end{array}} \text{(LOAD)}$$

$$\frac{\text{idx}, \text{load}_i \in \text{BoogieLocalVars} \quad \text{load} \in \{\text{load8_s/u}, \text{load16_s/u}, \text{load32_s/u}\} \quad e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\vdash e \text{ load} \hookrightarrow \begin{array}{l} \chi; \\ \text{call popToTmp1}(); \\ \text{idx} := \text{real_to_int}(\$tmp1); \\ \text{call load_i} := \text{mem_read_load}(\text{idx}); \\ \text{call push}(\text{int_to_real}(\text{load_i})); \end{array}} \text{(LOAD-EX)}$$

$$\frac{\text{idx} \in \text{BoogieLocalVars} \quad \text{store} \in \{\text{it.store}, \text{ft.store}\} \quad e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\vdash e_1 \text{ e}_2 \text{ store} \hookrightarrow \begin{array}{l} \chi_2; \\ \chi_1; \\ \text{call popToTmp1}(); \\ \text{call popToTmp2}(); \\ \text{idx} := \text{real_to_int}(\$tmp2); \\ \text{call mem_write}(\text{idx}, \text{real_to_int}(\$tmp1)); \end{array}} \text{(STORE)}$$

$$\frac{\text{idx} \in \text{BoogieLocalVars} \quad \text{store} \in \{\text{store8}, \text{store16}, \text{store32}\} \quad e_1, e_2 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\vdash e_1 \text{ e}_2 \text{ store} \hookrightarrow \begin{array}{l} \chi_2; \\ \chi_1; \\ \text{call popToTmp1}(); \\ \text{call popToTmp2}(); \\ \text{idx} := \text{real_to_int}(\$tmp2); \\ \text{call mem_write}(\text{idx}, \text{real_to_int}(\$tmp1)); \end{array}} \text{(STORE)}$$

$$\frac{e_1, e_2, e_3 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2 \quad \vdash e_3 \rightarrow \chi_3}{\vdash \text{memory.fill} \hookrightarrow} (\text{MEMORY.FILL})$$

```

χ3;
χ2;
χ1;
call popToTmp1();
call popToTmp2();
call popToTmp3();
call memory_fill(
    real_to_int($tmp3),
    real_to_int($tmp2),
    real_to_int($tmp1));

```

$$\frac{e_1, e_2, e_3 \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2 \quad \vdash e_3 \rightarrow \chi_3}{\vdash \text{memory.copy} \hookrightarrow} (\text{MEMORY.COPY})$$

```

χ3;
χ2;
χ1;
call popToTmp1();
call popToTmp2();
call popToTmp3();
call memory_copy(
    real_to_int($tmp3),
    real_to_int($tmp2),
    real_to_int($tmp1));

```

1.5.8 Table Operations

$$\frac{\text{idx} \in \text{BoogieLocalVars} \quad e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\vdash e \text{ table.get} \hookrightarrow \text{idx} := \text{real_to_int}(\$tmp1);} (\text{TABLE.GET})$$

```

χ;
call popToTmp1();
call $tmp1 := table_get(idx);
call push($tmp1);

```

$$\frac{idx \in BoogieLocalVars \quad e_1, e_2 \in exp_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\text{(TABLE.SET)}}$$

$\chi_1;$
 $\chi_2;$
 $\vdash e_1 \ e_2 \ \text{table.set} \hookrightarrow$
 $\begin{array}{l} \text{call popToTmp1()}; \\ \text{call popToTmp2()}; \\ \text{idx} := \text{real_to_int}(\$tmp2); \\ \text{call table_set}(\text{idx}, \$tmp1); \end{array}$

$$\frac{idx \in BoogieLocalVars}{\text{(TABLE.SIZE)}}$$

$\vdash \text{table.size} \hookrightarrow$
 $\begin{array}{l} \text{call idx} := \text{table_size()}; \\ \text{call push}(\text{int_to_real}(\text{idx})); \end{array}$

$$\frac{idx, load_i \in BoogieLocalVars \quad e_1, e_2 \in exp_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \vdash e_2 \rightarrow \chi_2}{\text{(TABLE.GROW)}}$$

$\chi_1;$
 $\chi_2;$
 $\text{call popToTmp1()};$
 $\vdash e_1 \ e_2 \ \text{table.grow} \hookrightarrow \text{call popToTmp2()};$
 $\text{idx} := \text{real_to_int}(\$tmp1);$
 $\text{call load_i} := \text{table_grow}(\$tmp2, \text{idx});$
 $\text{call push}(\text{int_to_real}(\text{load_i}));$

Explanation. Table instructions are translated using the Boogie runtime variables $\$table: [\text{int}] \text{real}$ and $\$table_size : \text{int}$. The instruction `table.get` reads an entry from the table and pushes it back onto the operand stack, while `table.set` updates the table at a given index. The instruction `table.size` pushes the current table size, and `table.grow` increases the table size and returns the previous size.

1.5.9 Function Calls and Locals

$$\frac{f \in FuncDecl \quad e_1, \dots, e_n \in exp_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \dots \quad \vdash e_n \rightarrow \chi_n}{\text{(CALL-F)}}$$

$\chi_1;$
 $\dots$
 $\vdash e_1 \ \dots \ e_n \ \text{call } f \hookrightarrow$
 $\begin{array}{l} \chi_n; \\ \text{call } f(); \end{array}$

$$\begin{array}{c}
\frac{\text{params}(f) = x_1, \dots, x_n}{\text{var arg1}, \dots, \text{argn} : \text{real};} \text{(FUNC-PROLOGUE)} \\
\vdash \text{func } f \hookrightarrow \\
\quad \text{var loc1}, \dots, \text{locm} : \text{real}; \\
\quad \text{call popArgsN}(\text{arg1}, \dots, \text{argn}); \\
\quad \text{loc1} := 0.0; \dots; \text{locm} := 0.0; \\
\\
\frac{x \in \text{LocalVars}}{\vdash \text{local.get } x \hookrightarrow \text{call push}(x);} \text{(LOCAL-GET)} \\
\\
\frac{x \in \text{LocalVars} \quad e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\vdash e \text{ local.set } x \hookrightarrow \begin{array}{l} \chi; \\ \text{call popArgs1}(x); \end{array}} \text{(LOCAL-SET)} \\
\\
\frac{x \in \text{LocalVars} \quad e \in \text{exp}_1 \quad \vdash e \rightarrow \chi}{\vdash e \text{ local.tee } x \hookrightarrow \begin{array}{l} \chi; \\ \text{call popArgs1}(x); \\ \text{call push}(x); \end{array}} \text{(LOCAL-TEE)} \\
\\
\frac{}{\vdash \text{return} \hookrightarrow \text{goto func_exit};} \text{(RETURN)}
\end{array}$$

Explanation. Function calls evaluate arguments left-to-right, push them, then call the Boogie procedure. Locals are assigned by popping values from the stack via auxiliary `popArgsN` procedures.

1.5.10 Indirect and Tail Calls

Unlike direct calls, indirect calls obtain the callee dynamically from a WebAssembly table. Since the target procedure cannot be resolved statically, SafeWasm conservatively models indirect calls by discarding the arguments and the table index, then pushing nondeterministic return values whenever the callee is expected to return results.

$$\frac{e_1, \dots, e_n, e_i \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \dots \quad \vdash e_n \rightarrow \chi_n \quad \vdash e_i \rightarrow \chi_i}{\text{(CALL-INDIRECT)}}$$

$\chi_1;$
 $\dots$
 $\chi_n;$
 $\vdash e_1 \dots e_n e_i \text{ call_indirect} \hookrightarrow \chi_i;$
 $\text{call popDiscard}_{n+1}();$
 $\text{for each expected result do}$
 $\quad \text{call push(nd_real());}$

$$\frac{f \in \text{FuncDecl} \quad e_1, \dots, e_n \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \dots \quad \vdash e_n \rightarrow \chi_n}{\text{(RETURN-CALL)}}$$

$\chi_1;$
 $\dots$
 $\vdash e_1 \dots e_n \text{ return_call } f \hookrightarrow \chi_n;$
 $\text{call } f();$
 $\text{goto func_exit};$

$$\frac{e_1, \dots, e_n, e_i \in \text{exp}_1 \quad \vdash e_1 \rightarrow \chi_1 \quad \dots \quad \vdash e_n \rightarrow \chi_n \quad \vdash e_i \rightarrow \chi_i}{\text{(RETURN-CALL-INDIRECT)}}$$

$\chi_1;$
 $\dots$
 $\chi_n;$
 $\vdash e_1 \dots e_n e_i \text{ return_call_indirect} \hookrightarrow \chi_i;$
 $\text{call popDiscard}_{n+1}();$
 $\text{for each expected result do}$
 $\quad \text{call push(nd_real());}$
 $\text{goto func_exit};$

Explanation. Direct tail calls are translated as ordinary Boogie procedure calls followed by a jump to the function exit label. In contrast, indirect calls cannot be resolved statically because the callee is obtained from a WebAssembly table. SafeWasm therefore adopts a conservative abstraction: all arguments together with the table index are discarded using the auxiliary `popDiscardN` procedure, and each expected return value is modeled by a fresh nondeterministic value produced by `nd_real()` and pushed onto the operand stack. This abstraction preserves soundness while avoiding explicit reasoning about dynamic function tables.

1.5.11 Global Variables

$$\begin{array}{c}
\frac{g \in GlobalVars}{\vdash \text{global.get } g \hookrightarrow \text{call push}(g);} \text{(GLOBAL-GET)} \\
\\
\frac{g \in GlobalVars \quad e \in exp_1 \quad \vdash e \rightarrow \chi}{\chi;} \text{(GLOBAL-SET)} \\
\vdash e \text{ global.set } g \hookrightarrow \text{call popToTmp1}(); \\
\quad g := \$tmp1;
\end{array}$$

1.5.12 Imported Functions

Imported WebAssembly functions have no implementation within the translated module. Therefore, SafeWasm translates each imported function into a Boogie stub procedure with the same signature. The procedure models an arbitrary external behavior using nondeterministic values.

$$\begin{array}{c}
\frac{f \in ImportDecl \quad \text{returns}(f) = \epsilon}{\vdash (\text{import } \dots (\text{func } f)) \hookrightarrow \text{procedure } fimport_i(\dots); \quad // \text{ no value is pushed}} \text{(IMPORT-VOID)} \\
\\
\frac{f \in ImportDecl \quad \text{returns}(f) = wt}{\text{procedure } fimport_i(\dots);} \text{(IMPORT-RETURN)} \\
\vdash (\text{import } \dots (\text{func } f)) \hookrightarrow \text{havoc result}; \\
\quad \text{call push}(\text{result});
\end{array}$$

Explanation. Imported functions are translated into Boogie stub procedures because their implementation is not available inside the translated module. If an imported function has no return value, the generated stub only models the external call and does not modify the operand stack. If it returns a value, SafeWasm nondeterministically generates a result using `havoc` and pushes it onto the stack. This provides a conservative abstraction of external behavior.

1.5.13 Global Declarations

Each WebAssembly global declaration is translated into a Boogie global variable. During module initialization, the initialization expression is evaluated and its resulting value is assigned to the corresponding global.

$$\begin{array}{c}
\frac{g \in GlobalDecl \quad \vdash e \rightarrow \chi}{\text{var } g: \text{ real};} \text{(GLOBAL)} \\
\vdash (\text{global } g \text{ wt } e) \hookrightarrow \chi; \\
\quad \text{call popArgs1}(g);
\end{array}$$

Explanation. Each global variable is represented as a Boogie global of type `real`. The initialization expression is translated and evaluated during module initialization, after which its value is assigned to the generated Boogie variable.

1.5.14 Exports

Export declarations only expose functions or globals to the execution environment and do not modify the operational semantics of the program. Consequently, they are ignored during the translation.

$$\frac{e \in \textit{ExportDecl}}{\vdash (\textbf{export} \dots) \hookrightarrow \epsilon} (\text{EXPORT})$$

Explanation. Export declarations have no effect on the verification semantics. Therefore, SafeWasm does not generate any Boogie code for exports. Only the selected entry procedure used for verification is preserved.

1.5.15 Verification Harness

The verification harness is generated after all WebAssembly functions have been translated into Boogie procedures. Let

$$F = \{f_0, \dots, f_{N-1}\}$$

be the set of translated functions of the module. SafeWasm generates an entry procedure that repeatedly selects one of these functions non-deterministically and invokes it with non-deterministically generated arguments.

	$F = \{f_0, \dots, f_{N-1}\}$	(BOOGIE-HARNESS)
$\vdash \text{Harness}(F) \hookrightarrow$	<pre> procedure BoogieEntry(); implementation BoogieEntry() { call initGlobals(); call InitRuntime(); while (true) { havoc c; assume 0 <= c && c < N; if (c == 0) { havoc and push arguments of f_0; call $f_0()$; discard return values; } else if ... else if (c == N-1) { havoc and push arguments of f_{N-1}; call $f_{N-1}()$; discard return values; } } } </pre>	

Explanation. The harness does not correspond to a WebAssembly construct. It is generated by SafeWasm to provide a verification entry point. Its purpose is to explore arbitrary sequences of calls to the translated WebAssembly functions. At each iteration, a function index is chosen non-deterministically, suitable arguments are generated and pushed onto the operand stack, and the selected function is invoked. Returned values are discarded after the call.

1.6 Discussion and Scope

The formal translation rules presented in this chapter constitute the core specification of the SafeWasm translation framework. They directly address the two fundamental challenges identified at the beginning of this chapter:

1. the **need for formal inference rules**, ensuring that the translation is precise, unambiguous, reproducible, and suitable for reasoning about correctness;
2. the **semantic mismatch between WAT and Boogie**. While WAT is a stack-based language in which instructions implicitly consume and produce operands on the execution

stack, Boogie follows a register-based, SSA-oriented programming model. Bridging this gap required the introduction of an explicit stack simulation together with auxiliary procedures for stack manipulation, numeric conversions, memory accesses, table operations, and the conservative modeling of imported and indirect function calls.

The rules presented in this chapter cover all instruction categories currently supported by SafeWasm, including constants, arithmetic and comparison operations, bitwise and numeric conversion instructions, memory and table operations, control-flow constructs, function calls, local and global variable manipulation, imported functions, and module-level declarations. Together, these rules formally specify how the implicit stack semantics of WebAssembly are translated into explicit Boogie code. Compared to previous work such as VeriSol [3], which translates Solidity into Boogie, our formalization must additionally capture the stack-based execution semantics of WebAssembly. This required the design of an explicit runtime model together with carefully defined helper procedures and auxiliary functions that faithfully preserve the operational behavior of WAT during translation. These aspects constitute one of the main contributions of this work

Bibliography

- [1] Mozilla Developer Network, “Webassembly documentation,” <https://developer.mozilla.org/en-US/docs/WebAssembly>, 2024, accessed: 09/2025.
- [2] K. R. M. Leino, “This is boogie 2,” Microsoft Research, Tech. Rep., 2008. [Online]. Available: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/krml178.pdf>
- [3] Y. Wang, S. Lahiri, S. Chen, R. Pan, I. Dillig, and C. B. andImmad Naseer, “Formal specification and verification of smart contracts for azure blockchain,” 2019. [Online]. Available: <https://www.microsoft.com/en-us/research/wp-content/uploads/2019/05/1812.08829.pdf>